Estadística Computacional
2do Cuatrimestre 2026
Trabajo Práctico

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Análisis estadístico de conteos de operaciones

Objetivo

El objetivo de este trabajo es estudiar estadísticamente el *costo* de algoritmos de ordenamiento, medido como la cantidad de operaciones elementales (comparaciones e intercambios) que realizan, y analizar cómo cambia en función del tamaño y del grado de desorden de la entrada.

Introducción

Una forma habitual de medir el costo de un algoritmo es cronometrar su ejecución. Sin embargo, el tiempo medido mezcla el costo del algoritmo con ruido ajeno a él (el sistema operativo, la memoria caché, otros procesos, la resolución del reloj), y ese ruido cambia de una computadora a otra y de una corrida a otra.

En este trabajo vamos a medir el costo *contando operaciones*: cada vez que el algoritmo compara dos elementos de la lista sumamos uno a un contador de *comparaciones*, y cada vez que intercambia dos posiciones sumamos uno a un contador de *swaps*. Estos conteos no dependen de la máquina y son exactamente reproducibles.

Pero entonces, ¿de dónde sale la aleatoriedad? Los algoritmos que vamos a usar son **deterministas**: con la misma lista de entrada hacen siempre exactamente las mismas operaciones. El azar entra **por la entrada**: si la lista a ordenar es elegida al azar, la cantidad de operaciones es una **variable aleatoria**. Una ventaja importante respecto de los tiempos es que para varios de estos conteos **la distribución verdadera es conocida**, de modo que vamos a poder contrastar nuestras estimaciones, intervalos y tests contra el valor real.

Algoritmos

Vamos a trabajar principalmente con *insertion sort* y con *quicksort*.

Algoritmo 1 Insertion sort instrumentado

```
1: función InsertionSort(a[0..n − 1])
2:     swaps ← 0;  comp ← 0
3:     para i ← 1 hasta n − 1 hacer
4:         j ← i
5:         mientras j > 0 hacer
6:             comp ← comp + 1
7:             si a[j − 1] > a[j] entonces
8:                 intercambiar a[j − 1] y a[j];  swaps ← swaps + 1;  j ← j − 1
9:             si no
10:                salir del mientras
11:    devolver (swaps, comp)
```

Algoritmo 2 Quicksort instrumentado (pivote = primer elemento, partición de Lomuto)

```
1: función Quicksort(a[lo..hi])
2:     si lo ≥ hi entonces devolver
3:     p ← a[lo];  i ← lo
4:     para j ← lo + 1 hasta hi hacer
5:         comp ← comp + 1
6:         si a[j] < p entonces
7:             i ← i + 1;  intercambiar a[i] y a[j];  swaps ← swaps + 1
8:     intercambiar a[lo] y a[i];  swaps ← swaps + 1
9:     Quicksort(a[lo..i − 1]);  Quicksort(a[i + 1..hi])
```

El código base incluye además *selection sort* y una variante de quicksort cuyo pivote es el elemento del medio del subarreglo.

Entradas aleatorias

Las listas de entrada son permutaciones de $0, 1, \ldots, n-1$ (sin repetidos). Usaremos dos modelos de entrada:

- **Permutación uniforme:** las $n!$ permutaciones tienen la misma probabilidad.
- **Desorden controlado k:** se parte de la lista ordenada y se aplican k *transposiciones al azar* (en cada paso se eligen dos posiciones i, j uniformes e independientes y se intercambian). Con $k = 0$ la lista está ordenada; a medida que k crece la lista se parece cada vez más a una permutación uniforme.

Metodología

Entregamos un código base en Python (`codigo_base.py`) que implementa los algoritmos con sus contadores, los dos generadores de entradas y una función `medir` que corre varios algoritmos **sobre las mismas listas**. El archivo `codigo_base.py` es la única definición válida de los contadores: no modifiquen dónde se cuenta una comparación o un swap, porque cambiarían las distribuciones.

Al ejecutarlo (tarda alrededor de medio minuto) se obtiene un archivo `conteos.jsonl` con una medición por renglón:

```
{"experimento": "uniforme", "algoritmo": "insertion", "n": 100, "k": null,
 "lista": 0, "swaps": 2311, "comparaciones": 2406}
```

donde `experimento` indica a qué corrida pertenece la medición (`uniforme`: $N = 2000$ permutaciones con $n = 100$; `tamanos`: $N = 200$ permutaciones para cada n; `desorden`: $N = 300$ listas con $n = 100$ para cada k), `k` es `null` para permutaciones uniformes y `lista` identifica la entrada (todas las filas con el mismo `experimento`, `n`, `k` y `lista` corresponden a la misma lista). Cada grupo debe fijar su propia semilla.

Notación

Para una permutación uniforme de tamaño n llamaremos S_n a los swaps de insertion sort, C_n a sus comparaciones y Q_n a las comparaciones de quicksort con pivote en el primer elemento. $H_n = \sum_{j=1}^{n} 1/j$ es el n-ésimo número armónico.

Los parámetros se escriben con letras griegas y sus estimadores con la misma letra y un sombrero (o una tilde, si hay más de un estimador del mismo parámetro). Para $n = 100$ escribimos $\mu_S = \mathrm{E}(S_{100})$, $\sigma_S^2 = \mathrm{Var}(S_{100})$, y análogamente μ_Q, σ_Q^2. Por ejemplo, a partir de N listas, $\hat{\mu}_Q$ es la media muestral de las comparaciones de quicksort.

En todo el trabajo n es el tamaño de cada lista, N la cantidad de listas de una muestra, R la cantidad de repeticiones de un experimento de simulación y k el desorden de la entrada.

Coeficiente de asimetría. Para una variable aleatoria X con media μ y desvío σ, el coeficiente de asimetría es

$$\gamma_X = \frac{\mathrm{E}\left[(X-\mu)^3\right]}{\sigma^3}.$$

A partir de una muestra $x_1, \ldots, x_N$ con media $\bar{x}$ se estima con

$$\hat{\gamma}_X = \frac{\frac{1}{N}\sum_{i=1}^{N}(x_i-\bar{x})^3}{\left(\frac{1}{N}\sum_{i=1}^{N}(x_i-\bar{x})^2\right)^{3/2}},$$

que es lo que calcula `scipy.stats.skew` con sus opciones por defecto.

Resultados conocidos

Para quicksort con pivote en el primer elemento sobre una permutación uniforme se sabe (Knuth, *The Art of Computer Programming*, vol. 3) que

$$\mathrm{E}(Q_n) = 2(n+1)H_n - 4n \approx 2n\ln n, \qquad \mathrm{Var}(Q_n) = 7n^2 - 4(n+1)^2 H_n^{(2)} - 2(n+1)H_n + 13n,$$

donde $H_n^{(2)} = \sum_{j=1}^{n} 1/j^2$. Estas fórmulas se usan como dato; no es necesario demostrarlas.

Test estadísticos

Nos interesa usar los conteos para contrastar hipótesis como:

- La distribución de los conteos es normal.
- Dos algoritmos (o dos modelos de entrada) tienen el mismo costo medio.
- El exponente de crecimiento del costo medio con n es el que predice la teoría.

Regresión lineal

Para estudiar la *complejidad* usaremos regresión lineal simple sobre variables transformadas. Si el costo medio crece como una potencia, $\mathrm{E}(S_n) \approx A\, n^b$, tomando logaritmos se obtiene el modelo lineal

$$\ln S_n = \beta_0 + \beta_1 \ln n + \epsilon,$$

donde β_1 estima el exponente b. Para quicksort, en cambio, $\mathrm{E}(Q_n)/n$ crece linealmente con $\ln n$, y el modelo propuesto es

$$\frac{Q_n}{n} = \beta_0 + \beta_1 \ln n + \epsilon.$$

Tareas

Bloque 1: Estadística descriptiva y modelo

1.1. **Entradas y conteos.**

a) Una permutación uniforme de tamaño n es una lista elegida al azar entre las $n!$ permutaciones de $0, \ldots, n-1$, todas con la misma probabilidad. Con la función del código base que genera permutaciones uniformes, genere 6000 permutaciones de tamaño $n = 3$ y arme una tabla con la frecuencia de cada una de las 6 posibles. ¿Qué distribución tiene la frecuencia de una permutación fija? ¿De qué tamaño esperaría que fueran sus diferencias con 1000? ¿Lo observado es compatible con que las 6 sean equiprobables?

b) Los algoritmos son deterministas. Explique por qué, aun así, S_n, C_n y Q_n son variables aleatorias. Con los conteos del experimento `uniforme`, determine cuáles de los siguientes conteos varían de una lista a otra y cuáles son constantes: swaps y comparaciones de insertion sort, swaps y comparaciones de selection sort, comparaciones de quicksort. Justifique el valor de los conteos constantes. ¿Puede un conteo ser constante aunque la entrada sea aleatoria?

c) Con los conteos del experimento `desorden`, ¿qué valores toman S_{100}, C_{100} y Q_{100} sobre la lista ya ordenada ($k = 0$)? ¿Son variables aleatorias? Compare el valor de Q_{100} con los que se obtienen sobre permutaciones uniformes.

1.2. **Descripción de S_{100} y Q_{100}.** Con las $N = 2000$ listas del experimento `uniforme`:

a) Muestre las distribuciones de S_{100} y de Q_{100} con histogramas, diagramas de caja y gráficos cuantil-cuantil contra la normal. Calcule media, desvío estándar, mediana, intervalo intercuartil y coeficiente de asimetría $\hat{\gamma}$.

b) En la definición de γ_X, ¿por qué se eleva al cubo y no al cuadrado? ¿Por qué se divide por σ^3? ¿Cuánto vale γ_X si la distribución de X es simétrica alrededor de su media?

c) ¿Qué diferencias de forma observa entre S_{100} y Q_{100}? ¿Es consistente la posición relativa de la media y la mediana con el valor de $\hat{\gamma}$? ¿Alguna de las dos distribuciones se parece a una conocida?

1.3. **Un modelo para S_n.** Una *inversión* de una lista a es un par de posiciones $i < j$ con $a[i] > a[j]$. Usaremos como dato los siguientes hechos:

- S_n es igual a la cantidad de inversiones de la lista.
- Para $j = 1, \ldots, n$ sea V_j la cantidad de elementos *anteriores* a la posición j que son mayores que el elemento en la posición j, de modo que $S_n = V_1 + \cdots + V_n$. Si la lista es una permutación uniforme, las V_j son **independientes** y $V_j \sim \mathcal{U}\{0, 1, \ldots, j-1\}$.

a) Para $n = 3$, liste las 6 permutaciones de $0, 1, 2$ y, para cada una, el vector (V_1, V_2, V_3) y el valor de S_3. ¿Es consistente la tabla con el dato? Calcule $\mathrm{E}(S_3)$ y $\mathrm{Var}(S_3)$ directamente de la tabla.

b) Calcule $\mathrm{E}(V_j)$ y $\mathrm{Var}(V_j)$, y deduzca que

$$\mathrm{E}(S_n) = \frac{n(n-1)}{4}, \qquad \mathrm{Var}(S_n) = \frac{n(n-1)(2n+5)}{72}.$$

¿En qué paso usó la independencia? Verifique con el ítem a).

c) Compare μ_S y σ_S^2 con la media y la varianza muestrales del ítem 1.2. ¿De qué orden de magnitud esperaría que fuera la diferencia entre la media muestral y μ_S?

d) Simule $N = 2000$ valores de S_{100} directamente desde el modelo, sumando V_j uniformes independientes, sin correr ningún algoritmo de ordenamiento. Compare con los conteos de insertion sort mediante histogramas superpuestos y un gráfico cuantil-cuantil de una muestra contra la otra. ¿Qué esperaría ver si el modelo fuera correcto?

e) Usando la estructura de S_n, explique por qué es razonable que su distribución sea aproximadamente normal. ¿Por qué no hay un argumento análogo para Q_n?

1.4. **Comparaciones de quicksort.**

a) Compare la media y la varianza muestrales de Q_{100} con μ_Q y σ_Q^2, calculadas con las fórmulas de la sección *Resultados conocidos*.

b) Con el experimento `tamanos`, grafique en función de n el desvío relativo σ_{Q_n}/μ_{Q_n} teórico y su estimación con los datos. ¿Qué pasa cuando n crece? ¿Qué significa esto en términos de lo predecible que es el costo de quicksort para listas grandes?

Forma y fecha de entrega

- Arme una notebook con la resolución del TP. No arroje código o gráficos sin explicación: intercale celdas de texto (con ecuaciones si fuera necesario), utilice Markdown para dar formato a secciones y subsecciones, etc., redacte y describa sus resultados. Las derivaciones (por ejemplo, las de los ítems 1.3 y **??**) pueden hacerse en celdas de texto.
- Suba la notebook en el link habilitado a tal efecto (es suficiente con que lo suba un solo integrante del grupo). Fecha límite de entrega: